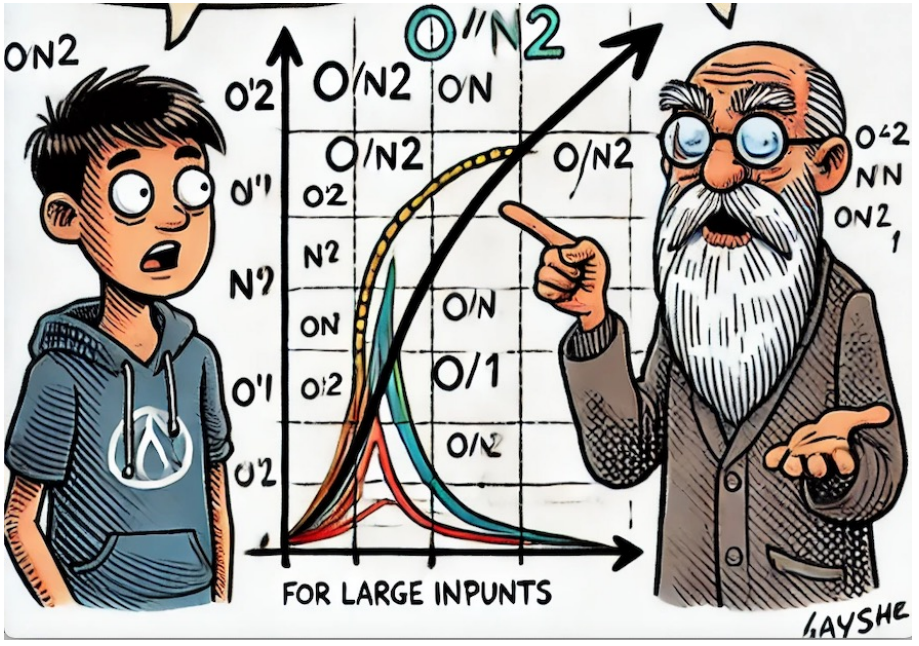
# Asymptotic Analysis “big-oh” notation

# Introduction to Asymptotic Analysis

- What is Asymptotic Analysis?

A method to describe the efficiency of algorithms based on their behavior as the input size grows.

- Compare algorithms by their growth rate, ignoring constant factors and lower-order terms.




- Real-World Analogy
- Planning a Trip:
- Short Distance: Walking vs. Biking – Minor time differences.
- Long Distance: Walking becomes impractical; Biking remains efficient.

# Asymptotic Analysis

Vocabulary for the design and analysis of algorithms (e.g., "big-Oh" notation).

High-level idea: Suppress constant factors and lower-order terms

too system-dependent

irrelevant for large inputs

Example: Equate $6n \log_2 n + 6$ with just $n \log n$.

Terminology: Running time is $O(n \log n)$

["big-Oh" of $n \log n$]

where $n$ = input size (e.g. length of input array).

# Example: One Loop (1/2)

Problem: Does array $A$ contain the integer $t$? Given $A$ (array of length $n$) and t (an integer).

**Algorithm 1**

```
1: for i = 1 to n do
2:     if A[i] == t then
3:         Return TRUE
4: Return FALSE
```

Question: What is the running time?

A) $O(1)$

B) $O(\log n)$

C) $O(n)$

D) $O(n^2)$

# Example: One Loop (2/2)

## Time Complexity Analysis: Why the Algorithm Runs in O(n)

**Algorithm 1**

```
1: for i = 1 to n do
2:     if A[i] == t then
3:         Return TRUE
4: Return FALSE
```

**The for-loop (Line 1):** The loop runs from i=1 to n, so it makes at most n iterations. Each iteration performs a constant-time check to see if the current element A[i] is equal to the target integer t.

**Best-case scenario (Line 3):** If t is found early in the array (say, at the first element), the algorithm returns TRUE, and the loop terminates early. However, this is the best-case scenario, which would result in a faster runtime (constant time in the best case), but Big-O notation describes the worst-case complexity.

**Worst-case scenario (Line 4):** In the worst case, the algorithm needs to go through the entire array (all n elements) and still doesn't find t. This leads to the loop iterating through all n elements before returning FALSE.

Thus, the time complexity in the worst case is proportional to the number of elements in the array, n. Therefore, the overall time complexity is O(n).

# Example: Two Loops

Given $A$, $B$ (arrays of length $n$) and t (an integer). [Does $A$ or $B$ contain $t$?]

**Algorithm 2**

```
1: for i = 1 to n do
2:     if A[i] = = t then
3:         Return TRUE
4: for i = 1 to n do
5:     if B[i] = = t then
6:         Return TRUE
7: Return FALSE
```

Question: What is the running time?

A) $O(1)$ C) $O(n)$

B) $O(\log n)$ D) $O(n^2)$

# Example: Two Loop (2/2)

## Time Complexity Analysis: Why the Algorithm Runs in O(n)

**Algorithm 2**

```
1: for i = 1 to n do
2:     if A[i] == t then
3:         Return TRUE
4: for i = 1 to n do
5:     if B[i] == t then
6:         Return TRUE
7: Return FALSE
```

**First for-loop (Lines 1-3):** The algorithm first checks array $A$ to see if any element equals $t$. In the worst case, the loop goes through all $n$ elements of $A$, making $n$ comparisons.

**Second for-loop (Lines 4-6):** If $t$ is not found in $A$, the algorithm proceeds to check array $B$. In the worst case, it will go through all $n$ elements of $B$, making another $n$ comparisons.

**Worst-case scenario (Line 4):** In the worst case, the algorithm needs to go through the entire array (all n elements) and still doesn't find t. This leads to the loop iterating through all n elements before returning FALSE.

Thus, the running time is linear with respect to the input size $n$, making it $O(n)$.

# Example: Two Nested Loops

Problem: Do arrays $A, B$ have a number in common? Given arrays $A, B$ of length $n$.

**Algorithm 3**

```
1: for i = 1 to n do
2:     for j = 1 to n do
3:         if A[i] = = B[j] then
4:             Return TRUE
5: Return FALSE
```

Question: What is the running time? Explain your answer.

A) $O(1)$ C) $O(n)$

B) $O(\log n)$ D) $O(n^2)$

# Example: Two Nested Loops

Problem: Do arrays $A, B$ have a number in common? Given arrays $A, B$ of length $n$.

**Algorithm 3**

```
1: for i = 1 to n do
2:     for j = 1 to n do
3:         if A[i] = = B[j] then
4:             Return TRUE
5: Return FALSE
```

Question: What is the running time?

A) $O(1)$ C) $O(n)$

B) $O(\log n)$ D) $O(n^2)$

# Example: Two Nested Loops

1. **Outer loop (Line 1):** The outer loop runs from $i = 1$ to $n$, meaning it iterates $n$ times, once for each element in array $A$.
2. **Inner loop (Line 2):** For each iteration of the outer loop, the inner loop runs from $j = 1$ to $n$, meaning it also iterates $n$ times. In each iteration, the algorithm checks if $A[i]$ is equal to $B[j]$ (Line 3).
3. **Comparison operation:** For each pair of elements $A[i]$ and $B[j]$, the algorithm performs a comparison to check if they are equal (Line 3). If a match is found, the algorithm returns $TRUE$; otherwise, it continues the loops.
4. **Worst case:** In the worst-case scenario (when there is no common element between arrays $A$ and $B$), the algorithm will perform $n \times n = n^2$ comparisons, as it needs to check all possible pairs of elements between the two arrays.

Thus, due to the two nested loops each running $n$ times, the total number of operations is proportional to $n^2$, resulting in a time complexity of $O(n^2)$.

# Example: Two Nested Loops (II)

Problem: Does array $A$ have duplicate entries?

Given arrays $A$ of length $n$.

**Algorithm 4**

```
1: for i = 1 to n do
2:     for j = i+1 to n do
3:         if A[i] == A[j] then
4:             Return TRUE
5: Return FALSE
```

Question: What is the running time?

A) $O(1)$ C) $O(n)$

B) $O(\log n)$ D) $O(n^2)$

# Example: Two Nested Loops (II)

Problem: Does array $A$ have duplicate entries?

Given arrays $A$ of length $n$.

**Algorithm 4**

```
1: for i = 1 to n do
2:     for j = i+1 to n do
3:         if A[i] == A[j] then
4:             Return TRUE
5: Return FALSE
```

Question: What is the running time?

A) $O(1)$ C) $O(n)$

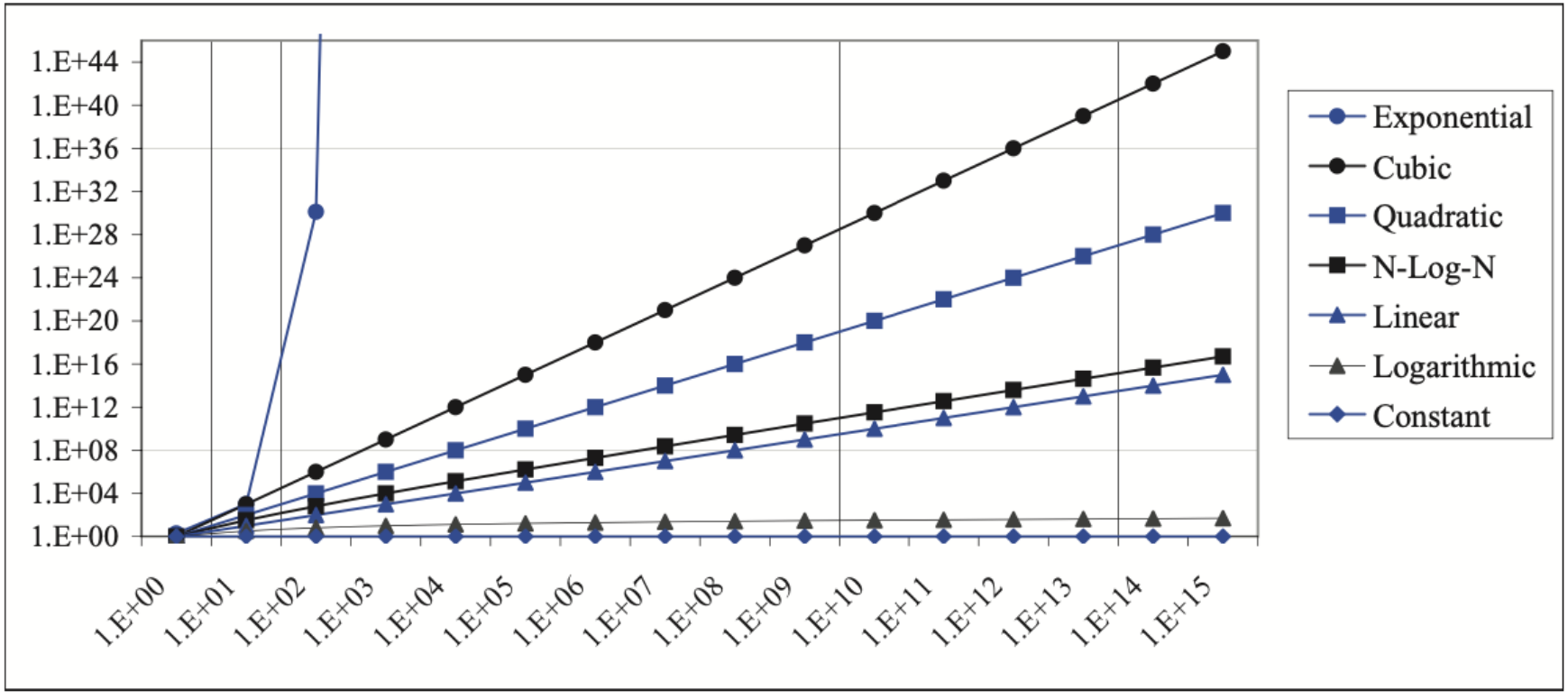
B) $O(\log n)$ D) $O(n^2)$

# Example: Two Nested Loops (II)

1. **Outer loop (Line 1):** The outer loop runs from $i = 1$ to $n$, meaning it will iterate $n$ times in total.
2. **Inner loop (Line 2):** For each iteration of the outer loop, the inner loop starts at $j = i + 1$ and runs up to $n$. So, on the first iteration of the outer loop, the inner loop runs $n - 1$ times, on the second iteration, it runs $n - 2$ times, and so on. This results in a total of roughly $\frac{n(n-1)}{2}$ comparisons.
3. **Overall complexity:** The algorithm performs a check (Line 3) during each iteration of the inner loop, and in the worst case, all the elements of the array are distinct, meaning both loops will run to their maximum capacity. This results in a quadratic number of operations, which gives us a time complexity of $O(n^2)$.

Thus, because there are two loops where the number of iterations of the inner loop depends on the current iteration of the outer loop, the running time grows quadratically with the size of the input, making the complexity $O(n^2)$.

# Comparing Growth Rate



# Comparing Growth Rate

**Exponential growth** (e.g., $2^n$): This grows the fastest, shooting up rapidly as n increases. An example is the rapid spread of a virus where each person infects two others.
**Cubic growth** (e.g., $n^3$): Grows slower than exponential but still accelerates significantly as n increases. This might represent the number of possible connections in a 3D grid.
**Quadratic growth** (e.g., n2): Slower than cubic, this could represent the number of edges in a complete graph with n nodes.
**N-log-N growth** (e.g., nlogn): Common in sorting algorithms like mergesort, this is more efficient than quadratic for large inputs but grows faster than linear.
**Linear growth** (e.g., n): Each step adds a constant amount, such as traveling in a straight line. Examples include simple loops that iterate over each element once.
**Logarithmic growth** (e.g., logn): This increases slowly even for large n. An example is binary search, where the search space is halved with each step.
**Constant growth** (e.g., 1): No matter the input size, the operation takes the same amount of time. An example would be accessing an element in a static array.

# HW Questions

# Question 1: Time Complexity Analysis

Consider the following algorithm:

```
for i = 1 to n:
    for j = 1 to n:
        if A[i] == B[j]:
            return True
return False
```

a) What is the time complexity of this algorithm? b) Explain why this is the case. c) Provide an example where the worst-case scenario occurs.

# Question 2: Time Complexity Analysis

Consider the following algorithm:

```
for i in range(n):
    for j in range(i+1, n):
        if A[i] == A[j]:
            return True
return False
```

a) What is the time complexity of the algorithm?
b) If $n$=1,000,000, estimate the number of comparisons performed in the worst case.

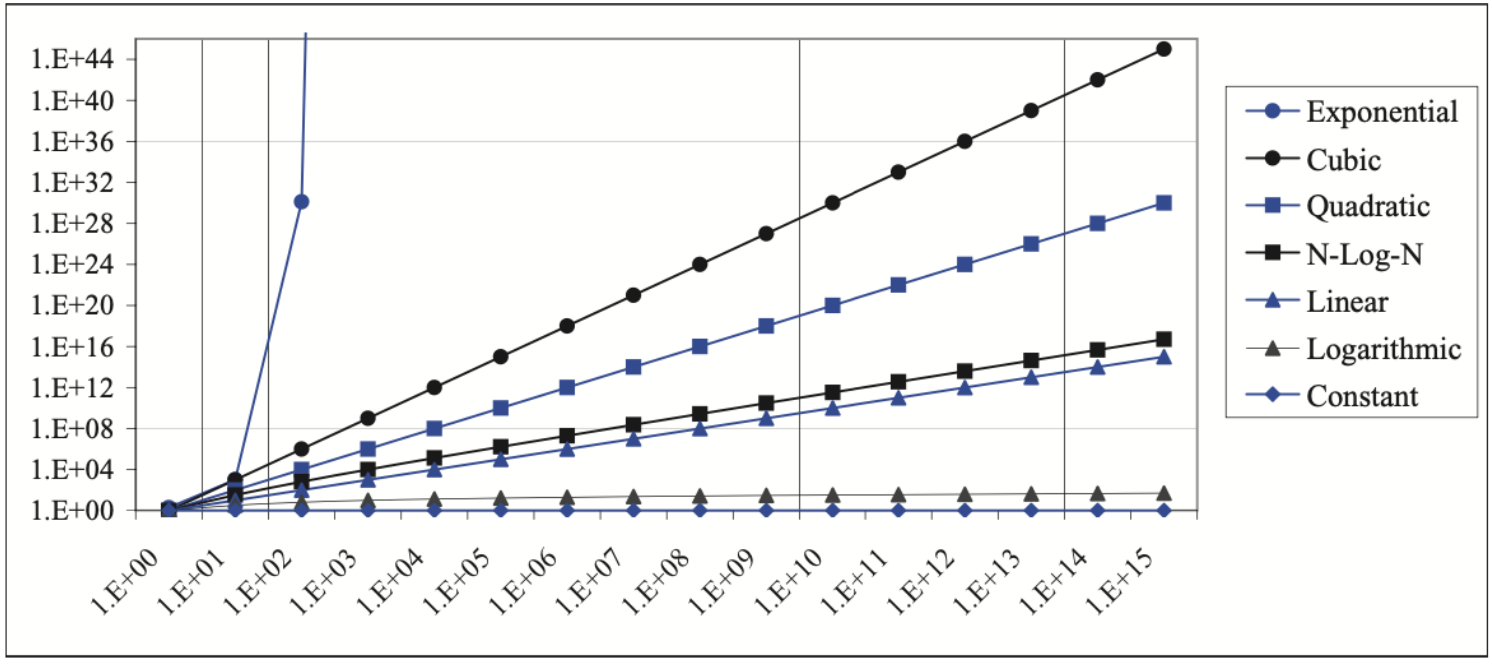
# Question 2: Growth Rate Comparison

Given the growth rates of algorithms as illustrated in the chart comparing exponential, cubic, quadratic, linear, and logarithmic growth:

Rank the following complexities from fastest growing to slowest:

$O(n^2)$, $O(2^n)$, $O(n \log n)$, $O(\log n)$

Which complexity would be preferable for large input sizes and why?



Next Class: Array